

Alternative splicing from nanopore cDNA

The workflow, and every tool in it

- Measuring splicing **without a reference transcriptome**
- *Arabidopsis* chr3 · six ONT cDNA libraries · **nucleus vs cytoplasm**, 3 each
- Two routes, side by side: **assembled from the reads**, and **curated AtRTDv2**

90 minutes · ~5 min compute · reads, references and binaries all bundled

Knowing the workspace

Opening a terminal:

```
$ mkdir ONT_workshop  
$ cd ONT_workshop  
$ git clone https://github.com/LongRead-UBA/LongRead-UBA-2026
```

Installing the enviroment:

```
$ cd ../ont-transcriptomics-workshop  
$ conda env create -f enviroment.yml
```

Opening the notebook:

```
$ conda activate ont-rna-3.1  
$ jupyter notebook  
Look for:  
/LongRead-UBA-2026/Practical\ sessions/ont-transcriptomics-workshop/  
notebook/3.1_Transcriptomics_splicing.ipynb
```

Nucleus vs cytoplasm: a known answer

Splicing happens in the nucleus

Only spliced mRNA is exported

So retained introns pile up in the nucleus

Why it matters

- A known answer tests the **method**, not the biology
- Get it backwards → the pipeline is broken
- So: does **de novo** reach the same conclusion?

Both routes agree: of the significant retained introns, **90 %** are nuclear on the de novo route, **98 %** on the reference.

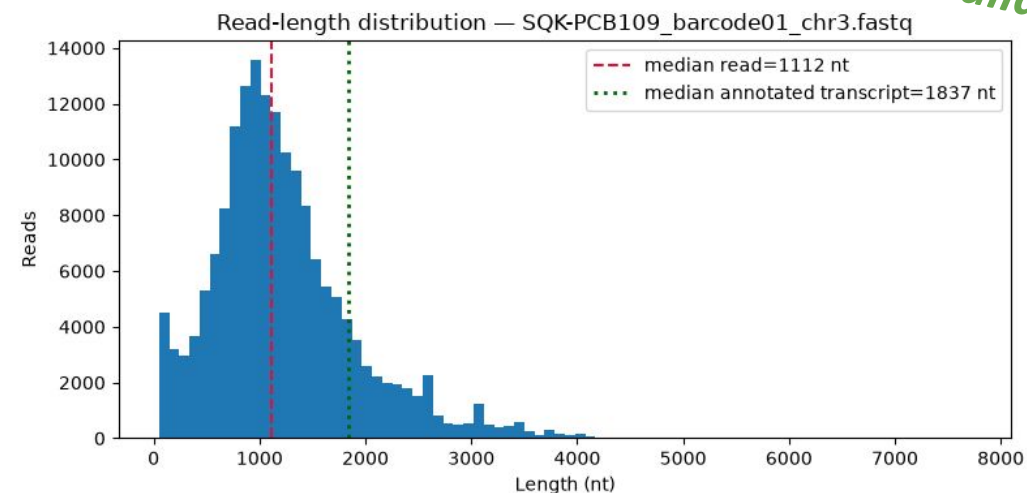
6 libraries, 2 compartments, 3 replicates

Already restricted to chromosome 3

Condition	Barcode	Reads	Mapped	Mapped %
Nuc_light	barcode01	174,670	171,796	98.4
Nuc_light	barcode02	113,395	112,004	98.8
Nuc_light	barcode03	30,431	29,996	98.6
Cit_light	barcode07	183,804	180,578	98.2
Cit_light	barcode08	114,175	111,212	97.4
Cit_light	barcode09	110,533	107,467	97.2

barcode03 is a 6.0× depth outlier. We surface it rather than hide it: assembly is depth-sensitive in a way quantification is not.

Read lengths vs transcript lengths



- 170,160 reads, **mean 1,230 nt**, median 1,112 nt, max 7,717 nt
- AtRTDv2 chr3: 10,524 models, **median 1,837 nt**
- **15 %** of reads reach the median transcript length; the median read is **61 %** of it

"Full-length cDNA" is an aspiration, not a guarantee — which explains most of the assembly artefacts we will see later.

The whole workflow

Four stages. Everything after stage 2 is identical for both routes — that is what makes the comparison fair.

The reads are aligned twice: to the genome in ①, then to each transcript set in ③.

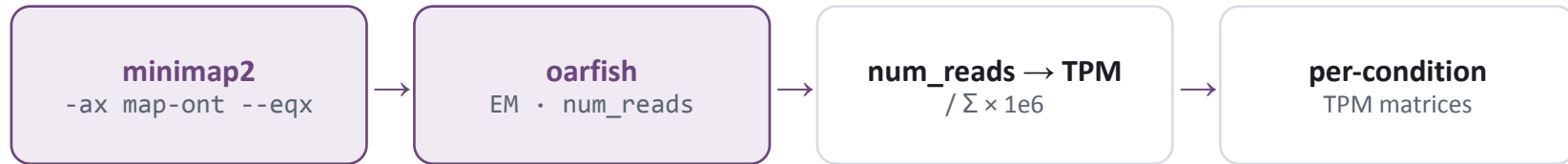
① Align



② Transcript set



③ Quantify



④ Splicing



Why long reads, and why two alignment passes

Why long reads suit splicing

- A short read sees **one junction at a time**; isoform structure has to be inferred
- An ONT cDNA read can span **many junctions at once** — the intron chain is **observed, not reconstructed**
- That is what makes de novo assembly credible here at all

...and what it costs

- Higher per-base error → splice sites land a **few bp off**
- Truncated reads → transcripts with the **right introns, wrong ends**
- Both show up in the assembly QC numbers, and neither is fatal for **event-level** analysis

Two alignments, two different questions

Pass 1 — to the GENOME

```
minimap2 -ax splice -uf -k14
```

Where are the introns? → assembly, QC, IGV

Pass 2 — to the TRANSCRIPTS

```
minimap2 -ax map-ont --eqx -N 100
```

Which isoform did this read come from? → oarfish

Pass 1 must be splice-aware (reads jump over introns). Pass 2 must NOT be — the transcript sequences already have the introns removed, so a spliced alignment there would be a bug.

The file formats

Five formats carry everything in this session. Each tool converts one into another. ALL this files are **text files** (except BAM - compressed)

FASTQ

.fastq

the reads, as sequenced

- **4 lines per read:** id, sequence, +, per-base quality
- 170,160 reads here · mean 1,230 nt · no positions yet

FASTA

.fna / .fa / .faa
.fasta / .fas / ..

sequence, with no alignment

- **2 lines per read:** >name header, then the bases. That is the whole format
- **Genome:** one record per chromosome (>Chr3)
- **Transcriptome:** one record per *transcript*, introns already spliced out

SAM / BAM

.sam / .bam

one row per alignment

- Where each read sits: reference, position, **CIGAR**
- SAM is text; BAM is the same thing compressed. .bai = index
- The CIGAR is where splicing becomes visible — an N is a skipped intron

GTF

.gtf

annotation: which transcripts exist

- **9 tab columns** + attributes; coordinates, never sequence
- transcript_id / gene_id say which exon belongs to what
- GFF3 is the same idea with ID= / Parent= instead

GFF3

```
Chr1 TAIR10 exon 3631 3913 . + . ID=exon:AT1G01010.1.1;Parent=mRNA:AT1G01010.1;gene_name=NAC001;Note=transcription factor;Dbxref=GeneID:837319
```

GTF

```
Chr1 TAIR10 exon 3631 3913 . + . gene_id "AT1G01010"; transcript_id "AT1G01010.1";
```

minimap2

The aligner. Used twice, with different flags.

What it does

- Seed-chain-align with minimizers; the **de facto standard** for long reads
- -k14 = smaller k-mer, tuned for **noisier ONT** reads
- -uf = find splice sites on the **forward transcript strand** — valid because SQK-PCB109 cDNA is **strand-aware**
- --eqx emits =/X instead of M, and -N 100 keeps up to 100 secondary hits

Why -N 100 matters. oarfish's EM can only weigh isoforms it is shown. Drop the secondary alignments and you have silently pre-decided the answer.

```
# pass 1 - genome, splice-aware
minimap2 -k14 -d index/genome.mmi genome.fna
minimap2 -t 8 -ax splice -uf -k14 \
    genome.mmi reads.fastq \
    | samtools sort -@ 8 -o sample.bam -

# pass 2 - transcripts, NOT splice-aware
minimap2 -t 8 --eqx -N 100 -ax map-ont \
    denovo_tx.mmi reads.fastq \
    | samtools view -b -o sample.tx.bam -
```

Do not coordinate-sort pass 2. oarfish needs alignments grouped by read, which is minimap2's natural output order. Sorting breaks it.

Cost: genome pass ≈ 50 s · transcriptome pass + quantification, both routes ≈ 2.3 min on 8 threads.

minimap2: what goes in, what comes out

INPUT

reads.fastq

the reads — 4 lines each, no positions

TAIR10_chr3.fna

FASTA · the genome, one record: >Chr3

denovo.fa / AtRTDv2.fa

FASTA · one record per transcript



OUTPUT

.sorted.bam

pass 1 · aligned to the genome

.bam

pass 2 · aligned to the transcripts

```
— one alignment from each pass — same reads, different reference
# pass 1, to the GENOME — a chromosome, and N = intron skipped
RNAME=Chr3 POS=461142 CIGAR=...233N...240N...79N...

# pass 2, to the TRANSCRIPTS — a transcript id, and no N at all
RNAME=MSTRG.1.1 CIGAR=2S 12=1X21=1I24= ...
```

The N operators are the splicing. Pass 2 has none.

§ 2 and § 5 · cell-08, cell-12

samtools

The plumbing — and our annotation-free source of truth.

What it does here

- `sort + index` → the coordinate-sorted BAM that **StringTie** and **IGV** both require
- `view -b` → SAM to BAM for the transcriptome pass (**unsorted**, on purpose)
- `flagstat` → the QC table: total, mapped, mapped %
- `view <region>` → **counting reads over one intron by hand**

```
samtools sort -@ 8 -o sample.sorted.bam -
samtools index sample.sorted.bam
samtools flagstat sample.sorted.bam

# read-level intron retention, no annotation:
samtools view sample.bam Chr3:462787-463031
#   spliced   = CIGAR N spans the intron
#   retained = read covers it with no gap
```

Every model we build gets checked against raw CIGARs

- SUPPA's PSI is a **model estimate** · a CIGAR N over the intron is **direct evidence**

samtools: what goes in, what comes out

INPUT

SAM on stdin

text, straight out of minimap2 — never written to disk

.sorted.bam

for flagstat and for counting by hand

OUTPUT

.sorted.bam + .bai

coordinate-sorted, with its index

flagstat text

a number you read — not a pipeline file

```
— flagstat, barcode01 — and why 'total' is not the read count
174670 + 0 in total (QC-passed reads + QC-failed reads)
170160 + 0 primary      <- this is the read count
  866 + 0 secondary     <- extra placements
 3644 + 0 supplementary  <- one read, split across two spots
```

Records, not reads: $170,160 + 866 + 3,644 = 174,670$.

§ 2–3 · cell-08, cell-10

StringTie

Builds a transcriptome from the reads, with no reference transcriptome at all.

What it does

- **-L = long-read mode:** treat each read as a near-full-length transcript instead of stitching short fragments through a graph
- Assemble **per sample**, then **--merge** the six GTFs into one transcript set
- **--merge** works on **transcripts, never on reads** — it takes the union and resolves compatible structures
- Output: `denovo.gtf`, genes named `MSTRG.*` — **no gene names, because nothing told it any**

	de novo (StringTie)	reference (AtRTDv2)
transcripts	8,210	10,524
genes	5,406	4,382
isoforms per gene	1.52	2.40
exon records	52,227	77,695

```
# one assembly per sample (6 x ~2 s)
stringtie -L -p 8 -o st_bc01.gtf bc01.sorted.bam

# union them into one transcript set
stringtie --merge -p 8 -o denovo.gtf st_*.gtf

# -L implies -s 1.5 -g 0 ; relative thresholds:
#   -f 0.01  min isoform fraction
#   -j 1     min junction coverage
```

Read the isoforms-per-gene row

- 1.52 vs 2.40: the assembly is **structurally poorer per gene**
- It drives everything downstream — an **RI event needs both** the spliced and retained forms to be annotated

"StringTie2 hasn't been updated in 7 years." 2019 is the *paper* date (Kovaka et al., *Genome Biology*). The binary here is v3.0.3, Nov 2025, commits into 2026. Last paper ≠ last commit — check the release history, not the citation.

StringTie: what goes in, what comes out

INPUT

6 × **.sorted.bam**

the genome alignments — **not** the FASTQs

(no annotation)

nothing tells it what genes exist

OUTPUT

6 × **st_<barcode>.gtf**

one assembly per library · STRG.* ids

denovo.gtf

the merged annotation · MSTRG.* ids

```
— two rows of denovo.gtf — coordinates, and no sequence anywhere
Chr3 StringTie transcript 8957 12462 1000 - . gene_id "MSTRG.1";
Chr3 StringTie exon 8957 9609 1000 - . transcript_id "MSTRG.1.1";

# the gap 9610-10250 between this exon and the next IS an intron,
# stated implicitly. A GTF lists exons; introns are what is left over.
```

Ids are not stable: per-sample STRG., after the merge MSTRG.*.*

§ 4 · cell-asm-code

Discuss: how should we pool and merge?

Write down your prediction before you look at the right-hand columns.

Strategy	commands	tx	iso/gene	precision	sensitivity	exact chain	RI events
A. per sample + merge	6 × -L, then --merge	8,210	1.52	53.9 %	69.7 %	31.3 %	884
B. per condition + merge	2 × -L on pooled BAMs	7,516	1.33	56.6 %	70.5 %	34.5 %	550
C. all 6 pooled once	1 × -L, no merge	9,569	1.66	58.8 %	69.7 %	34.7 %	2,442

All three cost 10–17 s. The demo retained intron survives all three.

The textbook argument — and why it fails here

- "Pool before assembly and a condition-specific transcript gets **averaged away**." That predicts **A wins**. It doesn't.
- --merge takes a **union** — it propagates each sample's artefacts instead of filtering them
- Junctions unique to each: **A 3,444 (11.6 % corroborated)** vs **C 1,138 (27.6 %)**

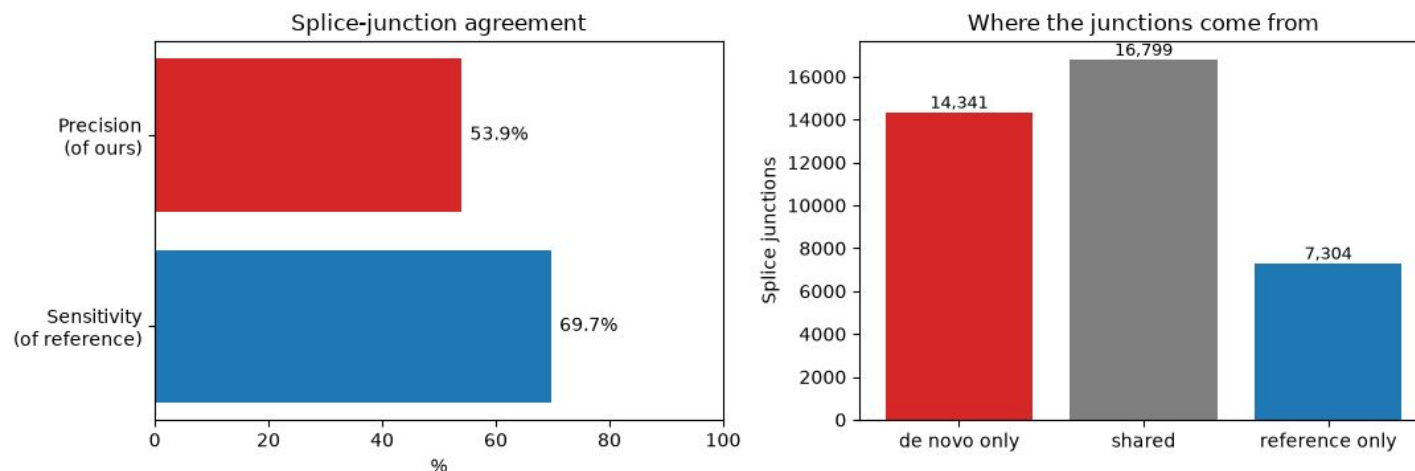
Why B collapses, and why we still ship A

- B has the fewest multi-isoform genes (**23.8 %** vs A 33.7 %, C 38.3 %) → RI events crash to 550
- We keep **A**: it is the community-standard idiom, it generalises to designs where pooling does hurt, and every downstream number here used it
- But it is **not** the best-scoring option on this dataset, and saying so is the lesson

The takeaway: "best practice" is a claim about a *typical* dataset, not a theorem about *yours*. This experiment took 15 seconds to run. Run it.

How good is that assembly?

Scoring the *de novo* junctions against AtRTDv2, which we treat as the known answer for chr3



The three numbers

- **53.9 % precision** — 14,341 junctions we propose that AtRTDv2 does not have
- **69.7 % sensitivity** — 7,304 annotated junctions we missed
- **31.3 % exact intron chain** — 2,530 / 8,088 multi-exon transcripts

How to read them

- AtRTDv2 is a "reference", not ground truth — some of our "false" junctions are real and unannotated
- Chain-level is always the lowest: **one wrong junction out of eight fails the whole chain**
- A truncated read yields the **right introns with a missing end exon** — fails an exact-chain test, still perfectly usable for a **local event**

The intron chain

A transcript's introns as an ordered tuple — its splicing pattern with the endpoints discarded. Endpoints depend on where the read happened to start and stop; the internal boundaries are real splice sites.

Same definition gffcompare uses for a structural match.

A price, not a disqualification. Every de novo result later inherits these numbers.

gffread

One command, one job: turn the assembled GTF into transcript sequences.

What it does

- Reads the GTF's exon records, **splices them out of the genome**, concatenates them in transcript order
- Handles the details you would get wrong by hand: **strand**, exon ordering, coordinate conventions
- Output: `denovo.fa` — **8,210 sequences, 15.2 MB**
- Runs in about **1 second**

```
gffread -w denovo.fa -g TAIR10_chr3.fna denovo.gtf
```

```
# -w write spliced transcript sequences (FASTA)  
# -g the genome to pull the exons from
```

From here the two routes are the same shape. A FASTA plus a GTF — the same kind of object AtRTDv2 is.

Why that matters: from here on, nothing can tell the difference

- oarfish needs transcript sequences to align against. It does not care who wrote them — **a curator or an assembler**.
- SUPPA2 needs a GTF in genome coordinates. Same again.
- So the two routes can share **identical code**: the annotation is a loop variable, and the comparison becomes a **property of the pipeline** rather than a claim in a slide.

gffread: what goes in, what comes out

INPUT

denovo.gtf

coordinates: which exons make each transcript

TAIR10_chr3.fna

FASTA · the sequence to cut those exons from

OUTPUT

denovo.fa

FASTA · 8,210 records, 15.2 MB

```
— coordinates in, sequence out — the same transcript, two ways
# in (denovo.gtf):  exon 8957-9609, exon 10251-10418, exon 10505-11095

# out (denovo.fa):
>MSTRG.1.1
TCTGACCAAAAATCTTCTGCACCAAGAAAAGTCTTATGATTGATCAATTAGAGA...
# ^ the three exons, spliced together. The introns are gone.
```

From here ours is a FASTA + GTF, exactly like AtRTDv2.

§ 4 · cell-asm-code

oarfish

Long-read transcript quantification by EM — and the reason salmon --ont is no longer an option.

What it does

- Reads the transcriptome BAM, runs **EM** to split ambiguous reads between compatible isoforms
- `--model-coverage` weights alignments by **where along the transcript** the read sits
- `--filter-group no-filters` — keep everything; we filter downstream instead
- Output columns: `tname`, `len`, `num_reads`. **There is no TPM column.**

```
oarfish -j 8 -a sample.tx.bam -o sample \
  --filter-group no-filters --model-coverage

# -> sample.quant : tname  len  num_reads
# all 10,524 transcripts appear, zeros included
```

num_reads → **TPM**: every EffectiveLength in the old salmon output was exactly 100.000, so `num_reads /  $\Sigma$  × 1e6` reproduces the same normalisation SUPPA2 was already being fed.

Why oarfish and not salmon — this is settled upstream, not a preference

	corr. with uniquely-mapped reads	violations of estimate $\geq$ unique reads
oarfish 0.10.3	0.881	0 / 1344 (0.0 %)
salmon 1.11.4	0.836	86 / 1344 (6.4 %)

- `salmon quant --ont` **no longer exists**: salmon 2.x removed long-read mode and prints a redirect to oarfish
- The old C++ line is frozen at 1.12.0 as `salmon-cpp`
- The two agree only **moderately** ($r \approx 0.63$ on log TPM) — **not** comparable with previous years' salmon numbers

The teaching point: an estimate must lie between a transcript's uniquely-mapped and total aligned reads. oarfish never violated that bound; salmon did, 86 times. **The newer model is measurably better behaved — not "same answer, new tool".**

oarfish: what goes in, what comes out

INPUT

.bam (transcriptome)
grouped by read — **must not** be coord-sorted

denovo.fa / AtRTDv2.fa
the transcript set it was aligned to



OUTPUT

<sample>.quant
TSV: tname, len, num_reads

<cond>_TPM.tsv
we build this — header is sample names only

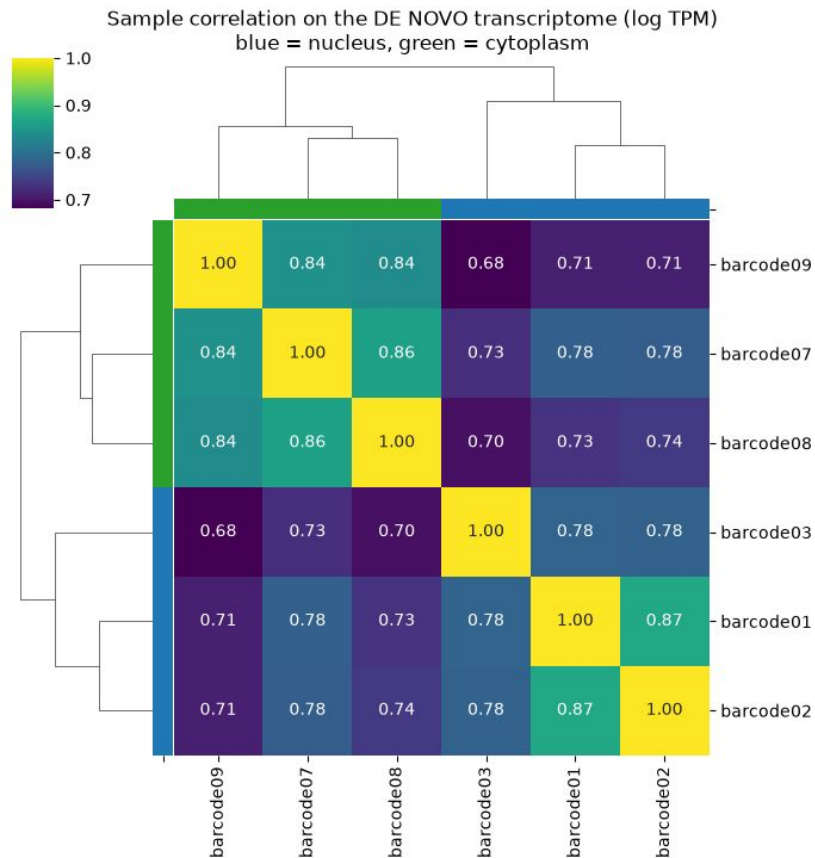
```
— one .quant file — three columns, and no TPM among them
tname      len      num_reads
MSTRG.1.1  2482    0
MSTRG.1.2  2515    2.763031634451571
MSTRG.2.1  2501    23.236968365548424
#          ^ fractional: the EM splits a read across isoforms
```

No TPM column, so we compute it: $\text{num_reads} / \Sigma \times 1e6$.

§ 5–6 · cell-12, cell-14, cell-16

From read counts to TPM

Before testing anything, check that replicates look like replicates.



What the matrix looks like

- de novo: **8,210** transcripts, 2,016 all-zero rows
- reference: **10,524** transcripts, 1,858 all-zero rows
- $\text{TPM} = \frac{\text{num_reads}}{\sum(\text{num_reads})} \times 1\text{e}6$, per sample

Within- vs between-condition correlation

route	within	between	separation
de novo (StringTie)	0.830	0.727	+0.103
reference (AtRTDv2)	0.769	0.657	+0.111

Both routes separate the compartments. What matters is the gap, not the absolute r.

If within $\approx$ between, stop.

SUPPA2

Turns transcript abundances into splicing measurements. Three subcommands, used twice each.

The three steps

- `generateEvents` — read the GTF, enumerate what **could** vary. Annotation only, no data.
- `psiPerIsoform` / `psiPerEvent` — combine with TPM to get **PSI**, the fraction of a gene's expression using that form
- `diffSplice --method empirical` — **Δ PSI** between conditions, with p-values from an empirical null built from the data itself

```
suppa.py generateEvents -i annot.gtf -o out -f ioi
suppa.py generateEvents -i annot.gtf -o out \
    -f ioe -e SE SS MX RI FL

suppa.py psiPerEvent -i events.ioe -e cond_TPM.tsv \
    -o cond_RI

suppa.py diffSplice --method empirical \
    --input events.ioe \
    --psi A.psi B.psi --tpm A.tsv B.tsv \
    --area 1000 --lower-bound 0.05 -gc -o cmp
```

Two levels, two different questions

-f ioi
isoform level
"which isoform
is used?"

-f ioe
local events
"which splicing
decision changed?"

Both levels: 372 vs 390 significant isoform changes, but the event view says which decision changed.

Three traps

- TPM header: sample names **only**
- `psiPerIsoform` → `X_isoform.psi`; `psiPerEvent` → `X.psi`
- Δ PSI = $PSI_2 - PSI_1$ — order matters

SUPPA2: what goes in, what comes out

INPUT

denovo.gtf / AtRTDv2.gtf
a GTF is all it needs — no sequence, no BAM

<cond>_TPM.tsv
one matrix per condition · transcripts × replicates



OUTPUT

.ioe / .ioi
the event definitions, from the GTF alone

.psi → .dpsi
PSI per event, then Δ PSI + p-value

```
— an event id is a coordinate, which is why routes can be compared
# events_RI_strict.ioe — one row per retained-intron event
MSTRG.20;RI:Chr3:70072:70711-70962:71389:-
#           ^chrom ^exon1 ^the intron ^exon2 ^strand

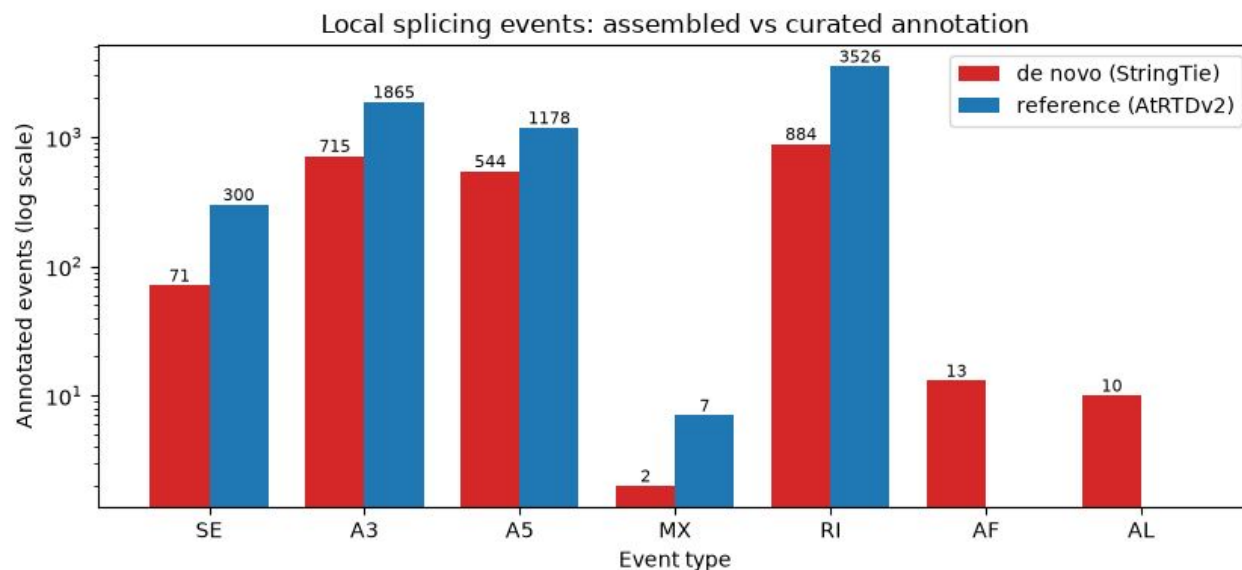
# .dpsi — the answer, per event
MSTRG.1013;RI:Chr3:2899214:... -0.4721800713 0.035964036
```

Events carry coordinates, so both routes match intron by intron.

§ 7–8 · cell-18, cell-22, cell-23

The seven local event classes

Same command, two annotations



	SE	A3	A5	MX	RI	AF	AL
de novo	71	715	544	2	884	13	10
AtRTDv2	300	1,865	1,178	7	3,526	0	0

- **RI dominates** — the class a nucleus/cytoplasm contrast predicts
- AF/AL: the **assembly finds 23, AtRTDv2 chr3 reports zero**. An assembly is a view of *your* sample.
- Every class is smaller on the de novo route, and the reason is the **1.52 vs 2.40 isoforms per gene** from earlier

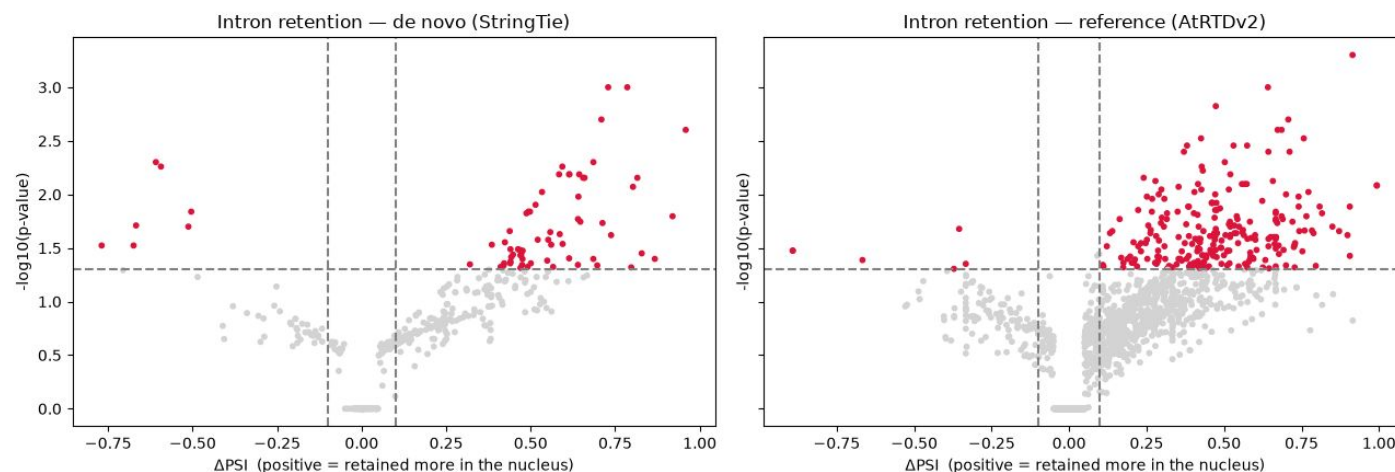
Why RI events are the ones to test here

- An RI event needs **both** forms present: the spliced transcript **and** the intron-retaining one
- So event count is driven by **isoform diversity per gene**, not by how many introns exist
- Retained introns mark **incompletely spliced nascent transcripts**
- Empty AF/AL files on the reference route are **expected output**, not a broken run

SE skipped exon · A3/A5 alternative 3'/5' splice site · MX mutually exclusive exons · RI retained intron · AF/AL alternative first/last exon

Differential intron retention, both routes

$|\Delta PSI| \geq 0.10$ and $p < 0.05$, empirical null



The column that matters

Both annotations put nearly all significant retained introns in the NUCLEUS — the direction predicted before anything was run.

The de novo route tests fewer events and calls fewer of them. It does not point the wrong way.

route	annotated RI	testable	significant	nucleus	cytoplasm	% nucleus
de novo	884	399	70	63	7	90.0 %
AtRTDv2	3,526	2,112	250	244	6	97.6 %

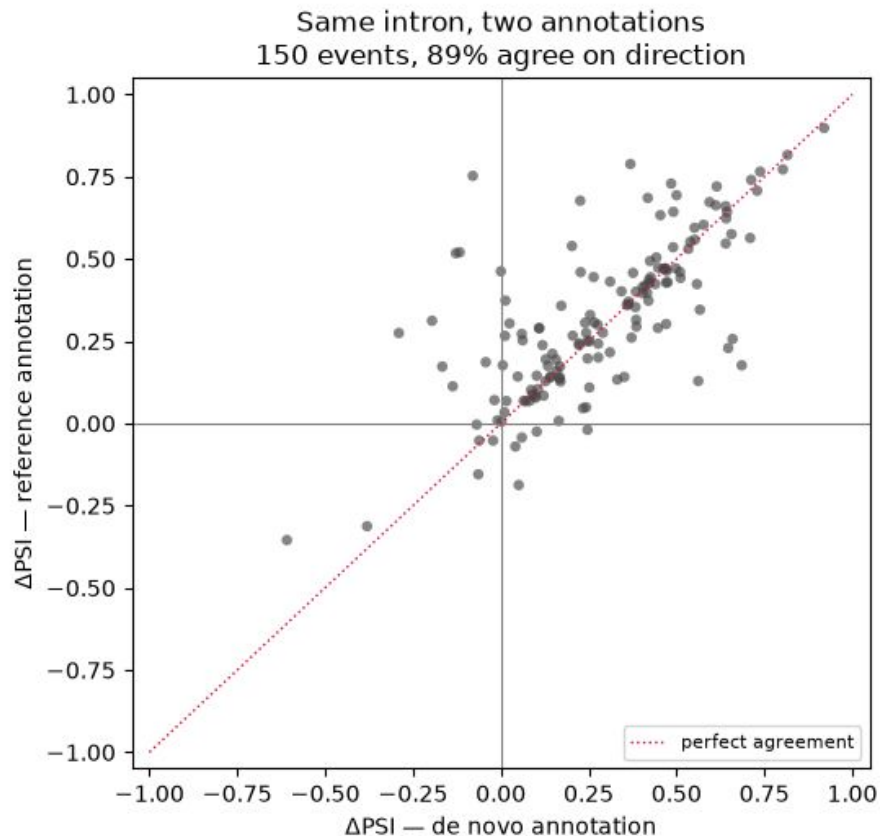
Isoform level, for contrast: 372 significant usage changes on the de novo route (of 3,593 tested) vs 390 on the reference (of 8,167). Similar headline counts — but only the event view tells you which splicing decision changed.

Follow the funnel

- **884** → **399**: an event is only testable if both forms are **expressed**, not just annotated
- **399** → **70**: significance
- The reference funnel is wider at every stage — which is what having a reference buys you

Do the routes agree on the same introns?

Not "do the counts match" — do they agree event by event, on introns both can see?



Matching by coordinate, not by transcript id

- An MSTRG.* id and AT3G02240 are different labels — so we match on the **intron's genomic coordinates**
- **150** introns are testable on both routes (of 399 de novo, 2,103 reference)
- Of **70** de novo hits, the reference could test **37**, and **also called 27 significant**
- → concordance where comparison is possible: **73 % (27/37)**
- On the 150 shared introns: Pearson **r 0.716**, Spearman **0.700**, and **89 % point the same way**

The naive figure would be 39 %. That divides by all 70 de novo hits — which counts "the reference had no opinion" as "the reference disagreed". Two different claims.

The plot to remember: the annotation changes *which* events you can see, far more than what *you conclude* about the ones you can.

IGV

The last step is always: look at the reads.

What it does

- Desktop IGV, opened on a session file the notebook generates from your own run
- Loads all six BAMs and both annotations over the same locus
- Every track needs its index beside it: .bai for a BAM, .fai for the genome FASTA
- Session files are generated, not shipped — they store absolute paths, so a shipped one only ever works on the machine that made it

Why this is the right way to end

- SUPPA's Δ PSI is a **model estimate**, three inferential steps from the reads
- The retention percentages on the right come from **counting CIGAR strings**
- **Same direction, different magnitude** — that contrast is the lesson

```
File → Open Session... → igv_sessions/<gene>.xml
```

or load it by hand:

```
# Genomes → Load Genome from File... TAIR10_chr3.fna
# File → Load from File... *.sorted.bam + both .gtf
```

Read-level evidence, annotation-free

	spliced	retained	retention
Nuc bc01	3	24	88.9 %
Nuc bc02	5	22	81.5 %
Nuc bc03	1	10	90.9 %
Cit bc07	7	0	0.0 %
Cit bc08	4	0	0.0 %
Cit bc09	3	0	0.0 %

For RNA reads, set the alignment track to Expanded and switch on Show splice junction track — retained introns are far easier to see uncollapsed.

IGV: what goes in, what comes out

INPUT

6 × .sorted.bam + .bai
the genome alignments from stage ①

TAIR10_chr3.fna + .fai
the reference the browser draws along the top

denovo.gtf · AtRTDv2.gtf
both annotations, plain text — no indexing needed



OUTPUT

the picture
reads you can look at — no output file

session .xml
what you open; IGV jumps straight to the intron

```
— a browser jumps to a locus, so it needs random access
samtools index sample.sorted.bam      # writes .bai  (done back in stage ①)
samtools faidx TAIR10_chr3.fna       # writes .fai

# .bai for BAM · .fai for FASTA · the GTF is read straight through
# no index -> IGV refuses the track, or draws nothing at all
```

It reads stage ① BAMs, not anything SUPPA made — an independent check.

§ 9 · cell-25, cell-28

The toolchain, end to end

All five command-line tools are bundled in tools/bin and run from an empty environment

Tool	Version	Role	Key invocation
minimap2	2.30	Align reads — twice, to genome then to transcripts	-ax splice -uf -k14 / -ax map-ont --eqx -N 100
samtools	1.21	Sort, index, QC; count reads over an intron by hand	sort · index · view -b · flagstat
stringtie	3.0.3	De novo transcript assembly from long reads	-L then --merge
gffread	0.12.9	Assembled GTF → transcript sequences	-w denovo.fa -g genome.fna
oarfish	0.10.3	Long-read quantification by EM	-a tx.bam --filter-group no-filters --model-coverage
SUPPA2	2.4	Events, PSI, differential splicing	generateEvents · psiPerEvent · diffSplice
IGV	desktop	Look at the reads; opens the generated session	File → Open Session... (session XML)

Python stack is not optional: SUPPA2 imports scikit-learn and statsmodels at start-up, so *every* subcommand fails without them — before it looks at your data.

Everything the notebook writes goes to results/. Delete that one directory to start clean. ~2.6 GB on a full run, of which 2.3 GB is BAMs.

What we did not do

Each is a choice with a cost

No reference-guided assembly.

We never passed AtRTDv2 to StringTie with -G. That would make the "de novo" route reference-guided — and then scoring its junctions against AtRTDv2 would be partly circular.

Only one assembler.

FLAIR and IsoQuant are both good and both maintained. Note that flair correct requires an annotation or short-read junctions to call novel splice sites — same circularity problem. We have not benchmarked them on this data.

No truly reference-free route.

RATTLE and isONform need no genome, but they emit sequences with no genomic coordinates, and SUPPA2's events require coordinates. A genome assembly is a hard requirement of this method — reference-transcriptome-free, not reference-free.

chr3 only, one contrast, three replicates.

That is what fits 90 minutes. It also means every number here is a chr3 number.

We kept the pooling strategy that scored worst.

Deliberately, and we said so. Community-standard idiom, and it generalises to designs where pooling does hurt.

Takeaways

1

You can measure alternative splicing without a reference transcriptome.

Both routes put ~90 %+ of significant retained introns in the nucleus. The de novo route is less sensitive, not wrong.

2

The annotation gates what you can see more than what you conclude.

73 % concordance where comparison is possible, 89 % agreeing on direction — but 150 shared testable introns out of 399 and 2,103.

3

Know what your number is made of.

Δ PSI is an estimate from an EM over an annotation you assembled. A CIGAR string with an N operator is evidence. Check one against the other.

4

Local events survive imperfect assemblies; full-length isoforms do not.

31.3 % exact-chain recovery sounds fatal and isn't — a truncated read has the right introns and the wrong ends, and events ask about one intron at a time.

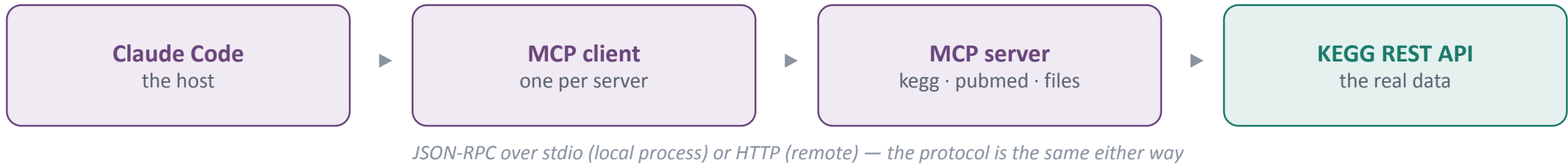
5

Measure your defaults instead of inheriting them.

The pooling experiment took 15 seconds and overturned the argument in every textbook. Paper dates are not release dates. Empty output files can be correct.

Extra: MCP servers and plugins

How an assistant like Claude Code reaches a database it was never trained on.



A server exposes three things

- **Tools** — functions the model may call
- **Resources** — data it can read
- **Prompts** — ready-made task templates

You still approve every call. A server widens what is reachable, not what runs unattended.

Why this matters in bioinformatics

- Live records instead of a **training cut-off**
- Ids resolved for you — no hand-copying accessions
- Local files and tools stay **on your machine**
- One question can cross KEGG, UniProt and PubMed

kegg-mcp-server — built for this kind of work: 34 tools, 9 resources, 4 prompts, no API key.

```
claude mcp add kegg-mcp-server -- uvx kegg-mcp-server # on PyPI, no key
```